Welcome!

COMP 3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 1: PYTHON FUNDAMENTALS

Agenda

1. Introduction
2. Logistics and Expectations
3. Intro to Python
4. Comments and docstrings
5. PEP-8 Style Guide
6. Strings, ASCII and Unicode in Python
7. Formatting output (str.format, f-strings)

A Bit About Me…

Jeffrey Yim (call me **Jeff**)

<u>Office Hours</u>:

By appointment on Discord

Mondays: 10:30am – 3:20pm

Email:

- **jyim3@bcit.ca**

Me

Education:

- Started at BCIT! w/Pascal
- Queen’s University
- Bachelor’s/Master’s Computer Science

Interests:

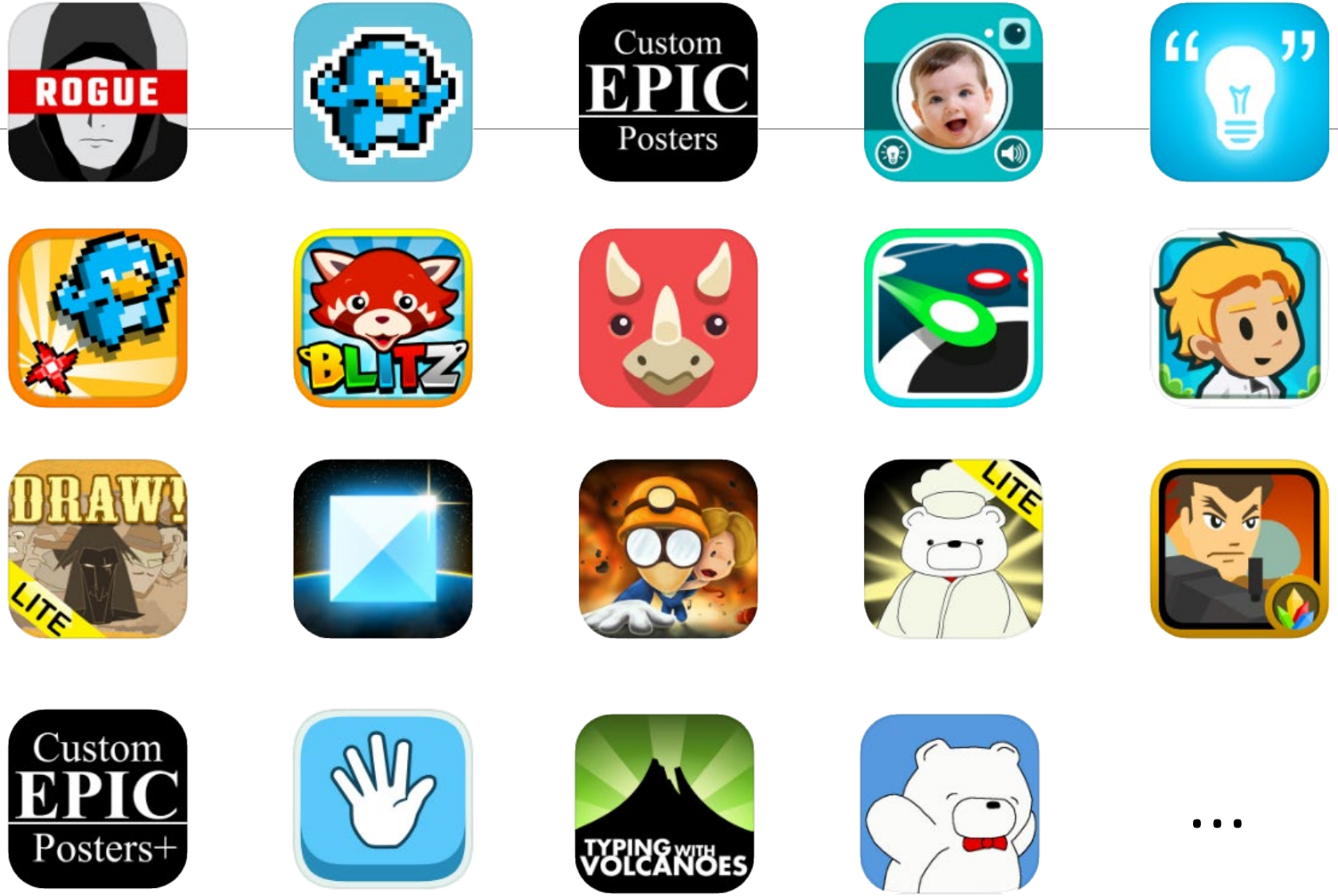
- Game development (COMP 7051)
- Unity
- Tesla
- Investing, urban kiz, DJing

Favorite language

- C#, Python is growing on me

Industry games

iOS games


ROGUE
Custom
EPIC
Posters
BLITZ
DRAW!
LITE
LITE
Custom
EPIC
Posters+
TYPING WITH
VOLCANOES

…

Let’s Get Through Some Administrative Stuff First

COMP 3522 - Three pillars of this course:

1. Learn how to **code in Python**
   - OOP concepts Classes, inheritance, polymorphism
   - Containers
   - Exceptions
   - etc

2. Learn how to **write good clean reusable code** using the S.O.L.I.D principles

3. Learn how to **write Design Patterns** – Behavioral, Creational, Structural patterns

Class Schedule

Lecture (All Sets)
Tuesday, 1:30PM – 4:20 PM

- Break every 50 minutes

Schedule

Lecture Breakdown:

1. Explaining topic and theory
2. Examine code
3. (Review videos) Each topic is a separate video on the learning hub

Schedule

Labs

Quiz at beginning of Lab

Try your best to be punctual

Time to work on lab and ask me questions

Attendance is taken during labs

- I need to report unapproved absences
- Unapproved absence of 10% or more of the labs may result in failure or forced withdrawal from this course

Labs due 11:59pm Friday

Schedule

Labs & assignments

-No late submissions

Submit code to github classroom

Michael (**Mike!** on discord) is our marker this term

Direct specific mark questions to him

Schedule

Midterm & Final exam

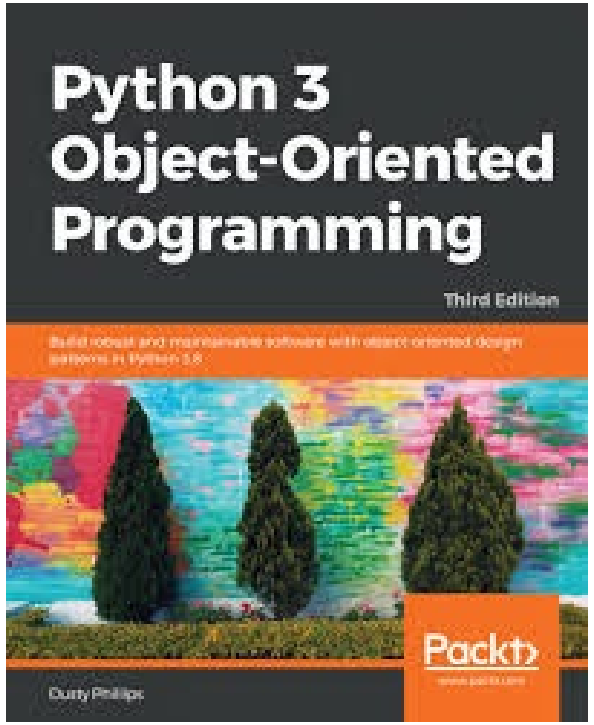
In-person hand-written paper exams

True/false, multiple choice

Short and long coding questions

Best way to practice is to complete the labs, assignments, and exercises in lectures

Textbook (Optional)



Philips, Dusty. (2018) ***Python 3 Object Oriented Programming*** *3rd Edition*. Packt Publishing;

- The text is available as an eBook via the BCIT Library and is a great supplemental read.
- There is a real shortage of tutorials and materials on advanced OOP in Python. This is a good one.

Online Resources

- https://www.geeksforgeeks.org/python-programming-language/
- https://www.tutorialspoint.com/python/index.htm

- Or search for specific topic:

1. “tutorialspoint python dunders”
2. “geeksforgeeks python dunders”

Resources: ChatGPT

You

You are my personal tutor for the python programming language

ChatGPT

Absolutely, I'd be happy to help you with Python programming! Whether you're a beginner or looking to advance your skills, feel free to ask any questions you have or let me know what specific topics you'd like assistance with. We can cover anything from

Use ChatGPT as your **personal tutor** to explain concepts and show sample code. *But don't always trust it*

Do not ask it to complete lab or assignment work for you

Do not submit ChatGPT code as your own

The Learning Hub

Will use for grades and posting lecture notes/labs/assignments

Github classroom

Will use for submitting labs/assignments

IDE

Pycharm

Communication

Communication with Discord!

- Check out the news post on the learning hub for the link

Use Discord for all communication. If it is something urgent, send me an email as well.

Sign up and change your nickname to your **first name and first letter of last name (Jeff Y)** on the server.

Responses may be delayed after school hours/weekends

Assessments

- Quizzes 10%
- Labs 10%
- Assignments 20%
- Midterm 30%
- Final exam 30%

To Pass:

50% overall
50% weighted average between the Midterm and Final Exam

Please aim higher (obviously)!

Let’s Lay Down Some Ground Rules…

- Be on time. Let me know if you can’t make it. Attendance will be taken each lab. (There is an attendance requirement).
- Be professional. Let’s make our class a safe space with no judgements.
- Assignments should be submitted on time. No late submissions (barring emergencies, medical reasons, or any such special circumstance).
- Plagiarism and cheating **won’t be tolerated**. If you submit work that is not yours, it **NEEDS** to be accredited. Work is to be submitted individually unless they explicitly require you to work in groups. If you are found in infringement of these policies, this will result in a grade of **ZERO**! (BCIT Plagiarism Policy: https://www.bcit.ca/files/pdf/policies/5104.pdf)
- That being said, don’t hesitate to discuss concepts introduced in class and help each other understand them. If you are in doubt, err on the side of caution. Ask me if you’re not sure!
- Support your peers and know when to ask for help! Your classmates are your support network and will be your contacts in the industry in the future. Be nice ☺

Let’s Lay Down Some Ground Rules…

- You are responsible for your own learning. Ask questions, come to office hours, reach out for assistance if you are falling behind and are stressed out.
- Phones should be on Silent and put away.
- Again, ask questions if in doubt! There are no stupid questions, and I will reiterate, this is a judgement free safe space. You are here to learn. If you want to chat privately or are outside of class, shoot me a message/email!
- Most importantly, **have fun!** Enjoy the learning process.

Intro to Python

My first Python program

```
print('hello world')
```

- Seriously, that's it!
- Notice there's **no semicolon ;**
- Python executes whatever code that's in the file, even without a main function. Code is interpreted top to bottom
- **Where's the main function???**

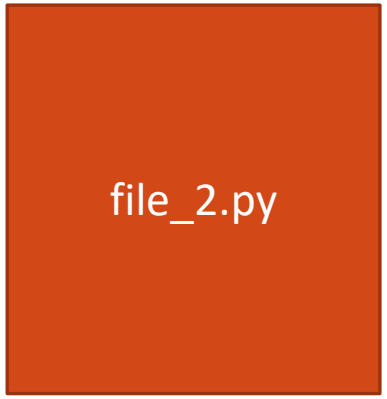
Where’s the main function???

__name__ is a built in variable in every python module (file)

Its value is set automatically

__name__ is set to the name of the current module

__name__ == ‘file_1’

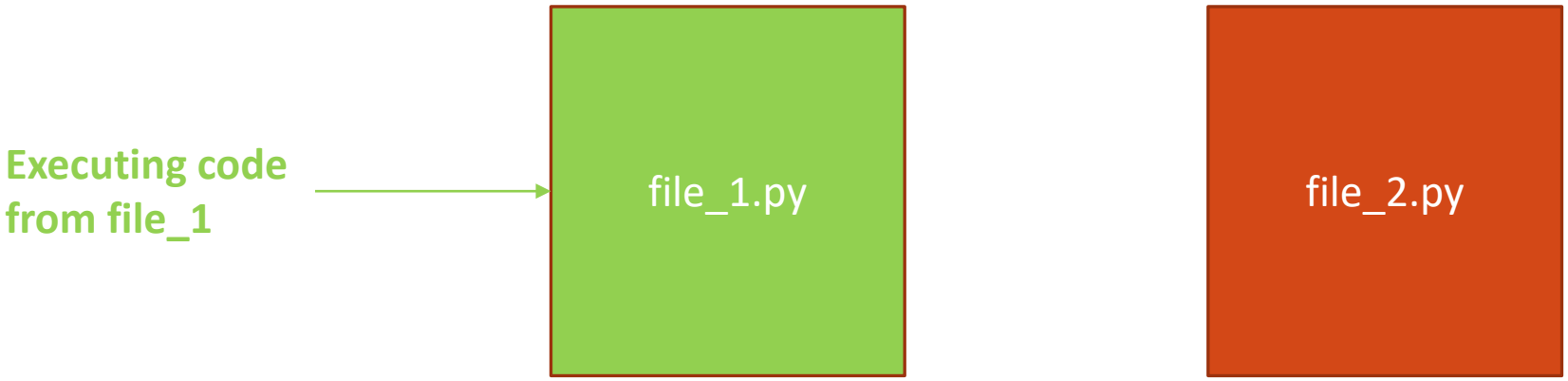


__name__ == ‘file_2’

Where's the main function???

But as soon as code begins running from a file, the **__name__** of that file evaluates to '**__main__**'

That file's name remains '__main__' for the duration of the program



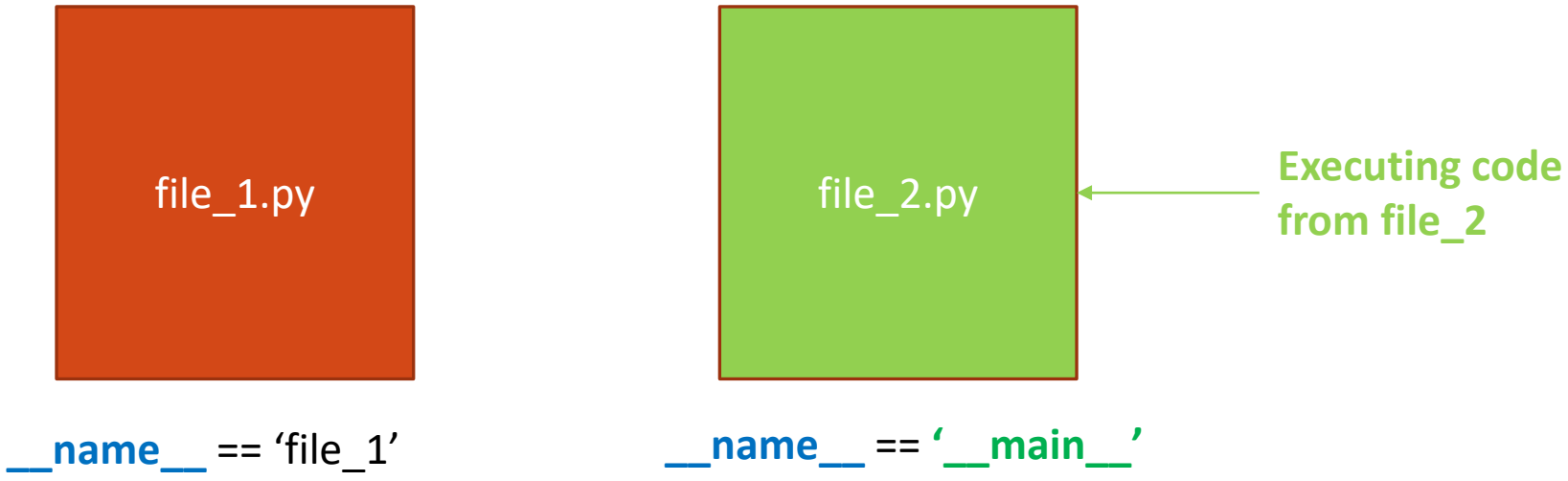
__name__ == '**__main__**'

__name__ == 'file_2'

Where’s the main function???

But as soon as code begins running from a file, the **__name__** of that file evaluates to ‘__main__’

That file’s name remains ‘__main__’ for the duration of the program

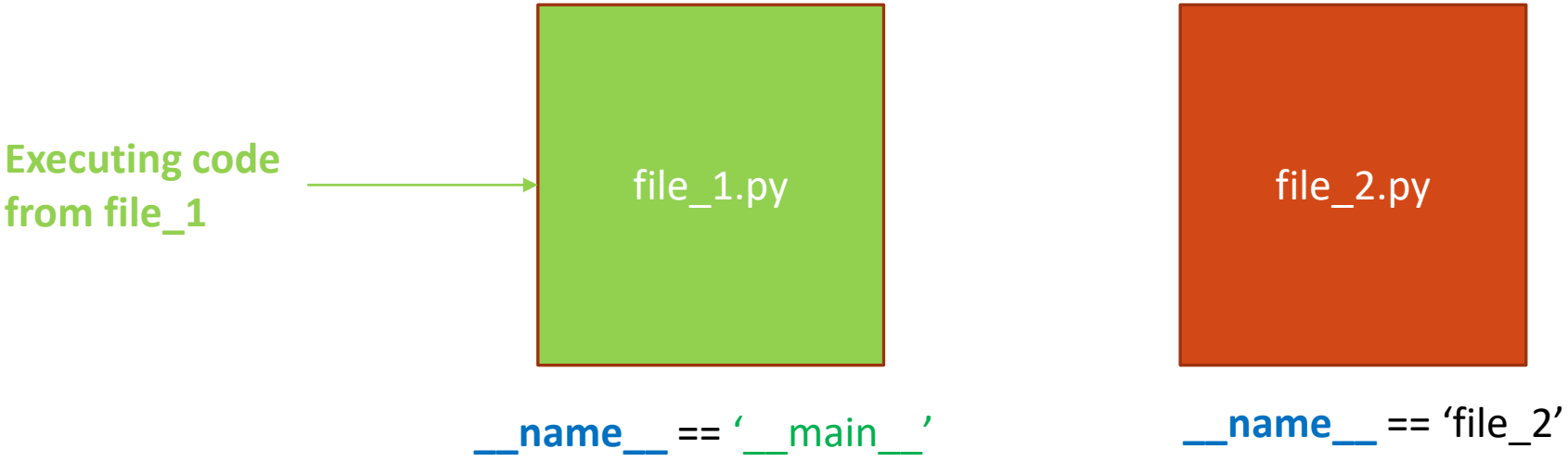


Where's the main function???

Python doesn't have a main function ☹ We'll need to 'hack' one in

Since we know **__name__** == '__main__' from the file we're running, we can 'hack' our code to have a main function

```
if __name__ == '__main__'
    main() #execute our main function
```



My first Python program

```
def main():
    print(‘hello world’)

if __name__ == ‘__main__’:
    main() #calls main function to print hello world
```

- This is the complete code to print out hello world, with a main function
- What this code is saying is “if we’re running code starting from this script, call a main function”

My first Python program

```
def jeff():
    print(‘hello world’)

if __name__ == ‘__main__’:
    jeff() #does the same thing as the previous slide
```

- Notice there’s nothing special about the naming of the main function
- I’ve changed the function to **jeff** and it behaves the same

file_1.py, file_2.py

Intro to Variables and Functions

Intro to Variables and Functions

```
def add(a, b):
    return a + b

num1 = input('Enter a number')
num2 = input('Enter a second number')
print(add(num1,num2))
```

- Function bodies begin after the **:**
- Notice how there are **no curly braces {}**
- Code sections are separated by **indentation**. Notice the indent before 'return a + b'
- That's not just for readability, python requires indentation to separate code sections

Intro to Variables and Functions

```
def add(a, b):
return a + b

num1 = input('Enter a number')
num2 = input('Enter a second number')
print(add(num1,num2))
```

Missing indent!

```
    return a + b
    ^
IndentationError: expected an indented block
```

- Compiler error if there's no indent for function. This applies for if/else, for, while etc
- Think of where you'd normally have curly braces in Java, and that's where you'd indent

Intro to Variables and Functions

```
def add(a, b):
    return a + b

num1 = 5
num2 = 10
print(add(num1,num2))
```

- Python is a dynamically typed language.
- Notice how there's **no type** (int) in front of num1 and num2. Types are inferred

Intro to Variables and Functions

```
def add(a, b):
    return a + b

num1 = 5
num2 = 10
print(add(num1,num2))
```

- Same as parameters in a function, notice how **types are not specified**

Intro to Variables and Functions

```
def add(a : int, b : int):
    return a + b

num1 = 5
num2 = 10
print(add(num1,num2))
```

- Can add : type after the parameter to indicate type
- But this is just for **programmer intention/readability** and it doesn't change functionality
- Can still pass in types other than int into function and it will work

Intro to Variables and Functions

```
def add(a, b):
    return a + b

num1 = 5
num2 = 10
print(add(num1,num2))
```

- Functions begin with def
- def is used in place of void or return types you're used to in Java
- No need to specify a function is void if it doesn't return anything, or a return type if it does return something

Intro to Variables and Functions

```
def add(a, b) -> int:
    return a + b

num1 = 5
num2 = 10
print(add(num1,num2))
```

- Can add -> type after the function header to indicate return type
- But this is just for **programmer intention/readability** and it doesn't change functionality
- Function can still return non-int types

add.py

If Statement

The if statement

```
if_stmt ::=  "if" expression ":" suite
             ("elif" expression ":" suite)
             ["else" ":" suite]
```

Branching: if

A branch in a program is **only taken if an expression's value is true**

This is known as an if-branch:

```
def calculate(salary, earned_bonus):
    if earned_bonus == True:
        salary = salary * 1.2
    return salary
```

Branching: if-else

An if-else structure has two branches:

1. The first branch is executed if the testing expression is true
2. The second branch is executed if the testing expression is false

```
def positive(number):
    if number < 0:
        return False
    else:
        return True
```

Be careful with your indentation!

```
def positive(number):
    if number < 0:
        print(”Negative”)
        return False
    else:
        print(”Positive”)
        return True
```

We can insert a **group (block) of statements** after if and else

Multi-branch if-else

```
if expression1:
    # Statements that execute when expression1 is true
    # (first branch)
elif expression2:
    # Statements that execute when expression1 is false
    # and expression2 is true
    # (second branch)
else:
    # Statements that execute when expression1 is false
    # and expression2 is false
    # (third branch)
```

Another example

```
password = input("Enter your password: ")

if password == "1234":
        print("Unacceptable password")
elif password == "abcd”:
        print("Dude, come on")
else:
        print("Thank you")

print("End of program")
```

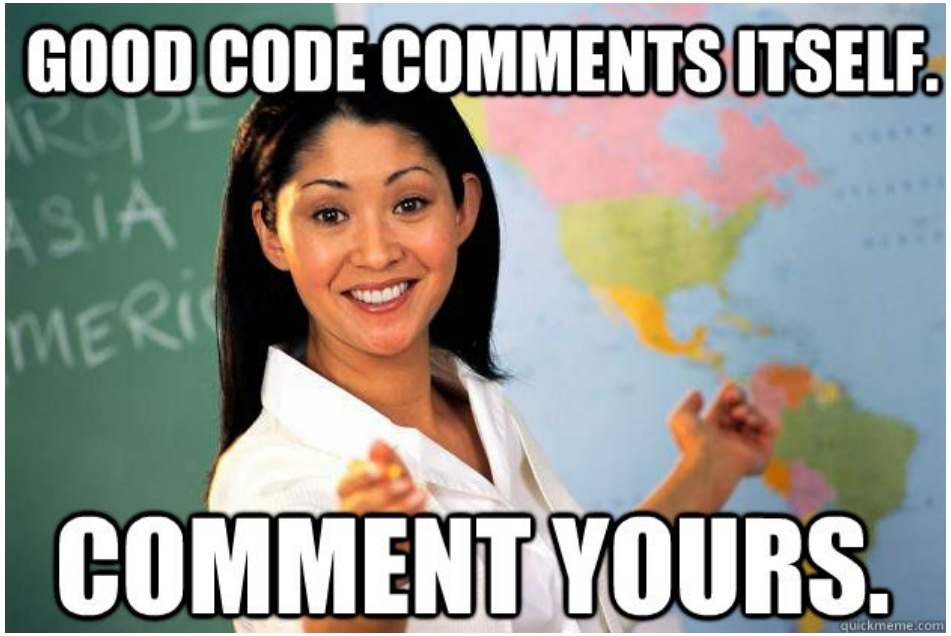
Comments and Docstrings

Good Comments & Readable Code

- Why do we comment?
- We strive to make our code self-documenting by using descriptive identifiers for functions, variables, etc.
- Good code should be self explanatory (right?)
- Python Comments
  - In line comments that start with **#**
  - Docstrings. Think Javadoc

// single line comment in java

single line comment in python



Docstrings

A special kind of comment

A string literal that describes a chunk of code

Implemented as the first statement inside a:

- Module (file)
- Function
- Class
- Method definition.

Used to create “official” documentation

Wrap with triple-quotes to span multiple lines

Python can use this to generate documentation

Docstring Example

```
def add_ints(a : int, b : int):
    """
    Return the sum of the two arguments.

    :param a: an int
    :param b: an int
    :precondition: a must be an int
    :precondition: b must be an int
    :return: the sum of the ints
    """
    return a + b
```

Docstring Formatting

Ensure you use a phrase that ends in a period.

Describe the function's effect as a command, i.e., '''**Do** this.''' or ''' **Return** that.'''

A multi-line docstring must start with a one-line summary, followed by a blank line, followed by the details of the description.

Suppose we have a program called my_module.py. The first statement in the file must be a triple-quote wrapped string, which will become the my_module.py module's docstring when the file is imported and used by another project.

```
>>> import my_module
>>> help(my_module)
>>> help(my_module.my_function)
```

my_module.py, calling_my_module.py

Styling Your Code

FASHION POLICE

PEP 8 Style Guide

https://www.python.org/dev/peps/pep-0008/

- The defacto accepted standard for formatting your Python code.
- At some point everyone here should go through it.
- There are a lot of rules but it eventually becomes second nature.
- Another website that summarized the PEP 8 guide (An easier read): https://realpython.com/python-pep8/

Why does this matter?

PEP 8 Style Guide

Consistency

Readability

Parsing

Efficient and Ease While Coding

Collaboration

Type	Naming Convention	Examples
Function	Use a lowercase word or words. Separate words by underscores to improve readability.	**function, my_function**
Variable	Use a lowercase single letter, word, or words. Separate words with underscores to improve readability.	**x, var, my_variable**
Class	Start each word with a capital letter. Do not separate words with underscores. This style is called upper camel case (PascalCase).	**Model, MyClass**
Method	Use a lowercase word or words. Separate words with underscores to improve readability.	**method, class_method**
Constant	Use an uppercase single letter, word, or words. Separate words with underscores to improve readability.	**CONSTANT, MY_CONSTANT, MY_LONG_CONSTANT**
Module	Use a short, lowercase word or words. Separate words with underscores to improve readability.	**module.py, my_module.py**
Package	Use a short, lowercase word or words. Do not separate words with underscores.	**package, mypackage**

There are a lot more rules that I won’t reiterate here. Check out the links!

Strings

Python String

A **string** is an immutable sequence (or string) of Unicode (UTF-8) code points

We can bind a string object to a variable

A **string literal** is the value of a string

We create a string literal using double- **or** single-quotes:

May contain letters, numbers, spaces, or symbols like @ or #.

"BCIT CST grads are the best"

'BCIT CST grads make the most money'

Python Strings are a Sequence Type

A string's letters (characters) are kept in order from first to last

A character's position in a string is called its index

The plural of index is indices

Indices start at 0

P	Y	T	H	O	N
0	1	2	3	4	5

An empty Python string has zero elements and is created like this:

```
my_empty_string = ''
```

Escape sequences

They're the ***same*** as everywhere else

Another way to print quotation marks is to **wrap the string around different quotes** as opposed to its content

Eg: print("That's skipping an unnecessary \' .")

```
print("hello'")
print('hello"')
```

Escape Sequence	Meaning
\'	Single quote (')
\"	Double quote (")
\t	Horizontal tab
\a	Bell (please don't do this)
\b	Backspace
\r	Carriage return
\n	New line (line feed)

Raw Strings

If we do not want to use escape characters we can create a **raw string**

A raw string is prefixed by the letter**r** or **R**

Python raw strings treat the backslash as a literal character

This is useful if we have a string that contains backslashes that should be interpreted as backslashes

Compare:

- `print(“Hello\nworld”)`

  ```
  Hello
  World
  ```

- `print(r“Hello\nworld”)`

  ```
  Hello\nworld
  ```

Some Common String Methods, Functions and Features

Try them out now in a python console

- “string”.title()
- my_string = “Comp 3522”
- my_string.upper()
- my_string.lower()
- len(my_string)
- my_string[3]
- my_string[0]
- my_string[-2]

Question: if we invoke these methods, does the original string change?

Some Common String Methods, Functions and Features

Question: if we invoke these methods, does the original string change?

NO!

Some Common String Methods, Functions and Features

1. capitalize
2. replace
3. isalpha
4. isdigit
5. split
6. join
   - this is faster than concatenation using += because no temp strings are created
   - This is an important and fast way to concatenate strings

https://docs.python.org/3.13/library/stdtypes.html#string-methods

REMEMBER: You can always type

- **help**(function name) to read the documentation
- **dir**(object) to read the properties and methods of the specified object

Eg:

```
help(str.isdigit)
print(dir(str))
```

String Concatenation

What happens when you do this?

```
pokemon_cast = “pikachu”
num_members = 150
print(pokemon_cast + “ and “ + num_members + “others”)
```

String Concatenation

What happens when you do this?

```
pokemon_cast = “pikachu”
num_members = 150
print(pokemon_cast + “ and “ + str(num_members) + “others”)
```

NOTE: Try casting num_members to a string

Multi-line strings

There are three ways we can do this:

1. Break lines with the \ character:

```
string_long = "This is a very long string" \
                  " that I wrote to help somebody" \
                  " who had a question about" \
                  " writing long strings in Python"
```

Multi-line strings

2. Use parentheses:

```
string_long = ("This is a very long string"
               " that I wrote to help somebody"
               " who had a question about”
               " writing long strings in Python”)
```

Multi-line strings

3. Use triple quotes:

```
string_long = """This is a very long string
                that I wrote to help somebody
                who had a question about
                writing long strings in Python"""
```

String Formatting

Formatting output

There are some easy ways to format output in Python

We can use:

1. the string format method
2. f-strings

It is always better to format than to concatenate! Change your habits if you tend to use ‘+’ to display data

1. The str.**format**() Method

The **format** method inserts values into strings

Replacement fields are delimited by curly braces:

```
“I earned {0} in {1}”.format(98.5, “Architecture”)
“I earned {} in {}”.format(98.5, “Architecture”)
“I earned {0} in {1} and {0} in {2}”.format(65, “Comm”, “Math”)
“I earned {grade} in {course}”.format(grade=65,
                                      course=“Comm”)
```

1. The str.format() Method

Eg: Try examples like these:

```
grades = (98.5, 65) #create tuple
print("I earned {g[0]} and {g[1]}".format(g = grades))

import math
print("Pi equals {m.pi}".format(m = math))
print("Pi equals {m.pi:.3f}".format(m = math))
```

2. F-strings

f-strings are string literals preceded by an **f** or **F** (like raw strings!)

Curly braces contain expressions that can be replaced with their values

Lets you drop in variable names directly in place

Evaluated at runtime:

```
grade = 98.5
course = “C”
print(f”I earned {grade} in {course}”)
name = “OOP 2 with Python”
print(F”My favourite course (so far) is {name.title()}!”)
```

A note on print()

There are different versions of the print( ) function:

1. print(‘Hello world’)
2. print(“Hello world”)
3. print(“Hello world”, **end=‘ ’**) #default end is newline
4. print(“I\’m ” **+** str(age) **+** “ years old”)
5. print(“I\’m”**,** age**,** “years old”)

You can learn more here:

https://docs.python.org/3.13/library/functions.html

Welcome!

COMP 3522 OBJECT ORIENTED PROGRAMMING 2

PART 2: PYTHON FUNDAMENTALS

Agenda

1. Mutability in Python
2. Sequence types
3. Lists
4. Tuples
5. Dictionaries
6. Iteration
7. Loops

Mutability

im·mu·ta·ble

/i(m)ˈmyo͞odəb(ə)l/

Adjective

From the Latin roots “in” (not) and “mutabilis” (to change)

unchanging over time or unable to be changed

String are Immutable

Writing or **altering** individual characters of a string variable is **not allowed**

Strings **cannot change once created**

Instead, an assignment statement must be used

```
word = ‘supercalifragilisticexpialidocious’

word = word.title( )  # this allocates new memory (and un-references the
previous one)

print(word) # ‘Supercalifragilisticexpialidocious’
```

Python Immutable Types

int( )

float( )

complex( )

str( )

tuple( )

bytes( )

frozenset ( )

Everything else is mutable (I think!).

Assignment Is Not Mutation

When we assign a value to a variable, we are actually assigning the reference to that value's location in memory

We will review this in some detail later in the term

For now, we can test this like so:

```
id(2.5)

a = 2.5

type(a)

id(a) # same address as id(2.5)!

a = a + 0.0456

id(a) # Not the same – a contains the address of a new float!
```

Memory.py

Sequence Types

PROBABLY THE THING PYTHON IS MOST FAMOUS FOR

Sequence Types

Sequence is the generic term for an ordered set

There are several distinct kinds of sequence types in Python

1. str (the string)
2. **list**
3. range
4. tuple
5. bytes

These are not the same thing.

They are stored differently and are treated differently, but they can be processed similarly

Only <u>some</u> sequence operations are common to all

Common Sequence Operations

Operation	Result
`x in s`	`True` if an item of *s* is equal to *x*, else `False`
`x not in s`	`False` if an item of *s* is equal to *x*, else `True`
`s + t`	the concatenation of *s* and *t*
`s * n` or `n * s`	equivalent to adding *s* to itself *n* times
`s[i]`	*i*th item of *s*, origin 0
`s[i:j]`	slice of *s* from *i* to *j*
`s[i:j:k]`	slice of *s* from *i* to *j* with step *k*
`len(s)`	length of *s*
`min(s)`	smallest item of *s*
`max(s)`	largest item of *s*
`s.index(x[, i[, j]])`	index of the first occurrence of *x* in *s* (at or after index *i* and before index *j*)
`s.count(x)`	total number of occurrences of *x* in *s*

* Well almost.

Range only supports item sequences that follow specific patterns, and hence doesn't support sequence concatenation or repetition

Mutable Sequence Operations

Operation	Result
`s[i] = x`	item *i* of *s* is replaced by *x*
`s[i:j] = t`	slice of *s* from *i* to *j* is replaced by the contents of the iterable *t*
`del s[i:j]`	same as `s[i:j] = []`
`s[i:j:k] = t`	the elements of `s[i:j:k]` are replaced by those of *t*
`del s[i:j:k]`	removes the elements of `s[i:j:k]` from the list
`s.append(x)`	appends *x* to the end of the sequence (same as `s[len(s):len(s)] = [x]`)
`s.clear()`	removes all items from *s* (same as `del s[:]`)
`s.copy()`	creates a shallow copy of *s* (same as `s[:]`)
`s.extend(t)` or `s += t`	extends *s* with the contents of *t* (for the most part the same as `s[len(s):len(s)] = t`)
`s *= n`	updates *s* with its contents repeated *n* times
`s.insert(i, x)`	inserts *x* into *s* at the index given by *i* (same as `s[i:i] = [x]`)
`s.pop([i])`	retrieves the item at *i* and also removes it from *s*
`s.remove(x)`	remove the first item from *s* where `s[i]` is equal to *x*
`s.reverse()`	reverses the items of *s* in place

List

Containers

A **container** is a data structure used to group related values together

A kind of compound type

A container contains references to other objects

A **list** is a container created by surrounding a sequence of variables or literals with square brackets **[ ]**

It's like an array, but since it's an object it has useful methods available. (It's similar to but better than Java's ArrayList)

List

This line of code creates a new list variable **my_list** that contains the two items, an integer and a string:

```
my_list = [10, 'abc’]
```

A list item is called an ***element***

A list is ordered and indexed. Just like arrays in other languages.

```
my_empty_list = []
```

A list can have many different types of elements. That is, its **heterogenous**.

```
student_data = [“A022343234”, [99,70,85.0], ‘P’]
```

List

Lists are useful for reducing the number of variables in a program (a single list can store an entire collection of related variables)

Individual list elements can be accessed by their index

A **list's index must be an integer**.

```
student_data = [“A022343234”, [99,70,85.0], ‘P’]
print(student_data[0]) #A022342334
```

Try this:

```
print(student_data[-1]) #P
print(len(student_data)) #3
```

Lists are Mutable

Let's try some of these list methods!

```
list_name.append(value)
list_name.pop(index) # remove and use value
list_name.remove(value) # remove by value
list() (Eg: spelling = list("ComplicatedWord"))
print(list_name)
list_name[1] = 'new updated value'
del list_name[3]  # remove by position
```

list_examples.py

```
list_name.sort()  # does this change the original list?
                    or does it return a new instance?

sorted(list_name) # how does this differ from list.sort?

len(list_name)

reverse(list_name)

dir(list)
```

list_examples.py

Sequence Functions and Methods

Sequence-type functions are built-in functions that operate on sequences like lists and strings

Examples include len( ), sorted( )

- `length = len(my_sequence)`
- `new_sorted = sorted(my_sequence)`

Sequence-type methods are methods built into the class definitions of sequences like lists and strings

Examples include sort( ), append( ), clear( ), insert( ), pop( ), remove( ), and reverse( )

- `my_sequence.sort()`
- `my_sequence.clear()`

What's the difference?

Membership Operator

A common task is to determine if a container contains a specific value

Python has membership operators **in** and **not in**

These operators return true if the left operand matches the value of some element in a container

```
colours = [‘red’, ‘white’, ‘cerulean’]
print(‘cerulean’ in colours) #true, cerulean exists in colours
print(‘magenta’ not in colours) #true, magenta doesn’t exist in colours
```

Membership Operators

Can be used with sequence types (string, list so far)

- We can check if a list contains something
- We can also use to determine whether a string is a **substring**, or a <u>matching subset of characters</u>, in a larger string:
- So much easier in Python compared to say C or C++

```
request_str = 'GET index.html HTTP/1.1'
if '/1.1' in request_str:
    print('HTTP protocol 1.1')

if 'HTTPS' not in request_str:
    print('Unsecured connection')
```

Tuples

Tuples

- Think Lists, but fixed and unchanging. That is, it is **immutable**.
- A tuple is also a sequence type, supporting len( ), indexing, and other sequence type functions

```
Eg: pos_vector = (3, 7.0, -39)
```

- Not commonly used, but great when element position and data is usually fixed and unchanging. Eg: location (longitude, longitude)

Tuples (Example)

```
parliament_hill_coords = (45.4236, 75.7009) #tuples use round parentheses
print('Coordinates:', parliament_hill_coords)
print('Tuple length:', len(parliament_hill_coords))

# Access tuples via index
print('\nLatitude:', parliament_hill_coords[0], 'north')
print('Longitude:', parliament_hill_coords[1], 'west\n')

# Error. Tuples are immutable
parliament_hill_coords[1] = 50
```

Dictionaries

Dictionaries

Similar to other languages, it is a **collection of key-value pairs**

Each key should be unique

```
dictionary_one = { ‘name’ : ‘Lwaxana Troi’ }
dictionary_two = { 1 : ‘COMP3522’,
                   2 : ‘COMP1712’, }
dictionary_three = { ‘Picard’ : ‘SP-937-215’ }
empty_dictionary = { }
```

What can we use for keys?

We do not want to be able to change the key once it is in the dictionary.

Data is accessed using a **key** instead of an index

Keys can be: Strings, numbers, tuples (**anything immutable**)

The keys in a dictionary do not have to be the same type, either!

```
my_dictionary = {} #create new empty dictionary using curly braces
my_dictionary['value'] = 100.0 #using string as key
my_dictionary[1] = 1 #using int as key
my_dictionary[2.5] = 'hello’ #using float as key
print(my_dictionary)
{'value': 100.0, 1: 1, 2.5: 'hello'}
```

Growing a dictionary

The dictionary, like the list, is a dynamic structure (it grows and shrinks as we add and remove key-value pairs):

```
my_character = { 'name' : 'Boaty McBoatface' }
print(my_character)
{'name': 'Boaty McBoatface’}
my_character['occupation'] = 'water bard'
print(my_character)
{'name': 'Boaty McBoatface', 'occupation': 'water bard'}
```

Dictionaries are mutable

We can delete key value pairs too

Use the del statement

```
my_dict = { 1: 'first', 2: 'second', 3: 'third', 3.5: 'third and a halfthth'}
print(my_dict)
{1: 'first', 2: 'second', 3: 'third', 3.5: 'third and a halfthth'}
del my_dict[3.5]
print(my_dict)
{1: 'first', 2: 'second', 3: 'third'}
```

We can use a dictionary in many ways

We can use it to store:

- key value pairs that represent the attributes of some single object, like a character

```
character = {
        ‘age' : 40,
        ‘height' : 100,
        ‘fav_food' : ‘donut’,
        ‘name' : ‘homer’,
}
```

We can use a dictionary in many ways

We can use it to store:

- a single piece of information about many kinds of objects, like this.

```
exam_results = {
        'abraham' : 50.0,
        'betty' : 60.0,
        'chuck' : 70.0,
        'dolorys' : 80.0,
        'edgar' : 90.0,
        'ferne mae' : 99.9
}
```

Iteration - Loops

For Loops

Used with sequence types

Will Loop over every element in the given sequence

Very efficient

```
for x in sequence:
    do something
```

Like other languages we can **break** and **continue**

For Loops

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
keys = dishes.keys()
values = dishes.values()

# iterate over values
n = 0
for val in values: #values contains [2, 1, 1, 500]
     n += val
print(n) #504
```

For Loops

What does this piece of code do?

```
for (name, age) in [(“James”, 22), (“Danica”, 35), (“Rohit”,28)]:
      if age > 30:
             print(“Name: {0}, Age: {1}”.format(name, age))
             break
```

For Loops

What does this piece of code do?

```
for val in "string":
    if val == "i":
        continue
    print(val)

print("The end")
```

For Loops

```
#the classic for loop, iterating over number range
for x in range(0,5):
    print(x)
```

```
Output:
0
1
2
3
4 #notice range does not include 5
```

While Loops

```
i = 1
while i < 6:
    print(i)
    i += 1
```

Use while loops when you need to loop over something that is not a sequence.

Remember to update the test expression!

Treat Python With Care

Python Pitfalls

- At first glance Python feels more like a scripting language without rigid rules.
- It's "Loosey Goosey"
- It is tempting to use Python to write bad code
- We will come across situations where this flexible nature of Python is an asset

That’s All For Week 1!

<u>Quizzes start next week!</u>

- The Quiz next week will be during your lab
- Next week we will also get into OOP concepts and all the good stuff!